University of Asia Pacific

Department of Computer Science and Engineering

CSE 402: Operating System Lab

LAB REPORT

Submitted by: Shad Bin Moshiur

Student ID: 23101143

Section: C-2

Lab task: 4

Submitted to:

Atia Rahman Orthi
Lecturer,
Department of Computer Science and Engineering

Date of Submission: 17/08/2026

LAB REPORT 5

SJF (Preemptive) CPU Scheduling

Operating System Lab | Student ID: 23101143

1. Objective

To implement the preemptive version of Shortest Job First (SJF) — also known as Shortest Remaining Time First (SRTF) — CPU scheduling algorithm in Python, and to compute the Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) for a given set of processes.

2. Theory

2.1 SJF Preemptive (SRTF)

In preemptive SJF, at every scheduling decision the CPU is assigned to the ready process with the smallest remaining burst time. Unlike non-preemptive SJF, a running process can be interrupted if another process with a shorter remaining burst time becomes ready. This minimizes average waiting time further than non-preemptive SJF but requires frequent re-evaluation of the ready queue and knowledge of remaining burst times.

2.2 Quantum-Based Variant Used in This Implementation

Rather than re-evaluating after every single time unit (the strict textbook definition), this implementation re-evaluates the ready queue and selects the process with the shortest remaining time, then runs that process for at most one time quantum (here, quantum = 2) before checking again. If the selected process still has remaining burst time after the quantum, it stays in contention and may be preempted by a shorter job at the next evaluation point. This reduces the number of context switches compared to strict SRTF while still preserving the "always run the shortest remaining job" priority rule at each decision point.

3. Process Set Used

The following process set (Arrival Time and Burst Time in ms) was used, with a time quantum of 2 ms:

PID	Arrival Time (AT)	Burst Time (BT)
P1	4	2
P2	2	2
P3	1	3
P4	0	6
P5	3	1

Table 1: Input process set (AT = Arrival Time, BT = Burst Time)

4. Implementation Summary

The algorithm was implemented as a single function, sjf_preemptive(processes, quantum):

- At each step, every arrived, unfinished process is scanned to find the one with the smallest remaining burst time.
- That process executes for min(quantum, remaining burst time) time units.
- If its remaining burst time reaches zero, it is marked complete and its Completion Time is recorded; otherwise it remains eligible for selection in the next round.
- If no process has arrived yet at the current time, the clock advances by one unit until a process becomes ready.

CT, TAT (= CT − AT), and WT (= TAT − BT) were then computed for each process, along with the average TAT and WT.

5. Execution Timeline (Gantt Chart)

Tracing the algorithm step by step for the given process set produces the following execution order:

Time Interval	Process Executed
0 - 2	**P4**
2 - 4	**P2**
4 - 5	**P5**
5 - 7	**P1**
7 - 9	**P3**
9 - 10	**P3**
10 - 12	**P4**
12 - 14	**P4**

Table 2: Order in which the CPU was allocated to each process

6. Results

PID	AT	BT	CT	TAT	WT
P1	4	2	7	3	1
P2	2	2	4	2	0
P3	1	3	10	9	6
P4	0	6	14	14	8
P5	3	1	5	2	1
Average	-	-	-	**6.00**	**3.20**

Table 3: SJF (preemptive) scheduling result

7. Program Output

The screenshot below shows the actual console output produced by running the script in Section 10:

```
•••   Process AT   BT   CT   TAT  WT
      P1      4    2    7    3    1
      P2      2    2    4    2    0
      P3      1    3    10   9    6
      P4      0    6    14   14   8
      P5      3    1    5    2    1

      Average TAT = 6.00
      Average WT = 3.20
```

Figure 1: Console output — SJF (preemptive) scheduling result

8. Discussion

P2 (AT = 2, BT = 2) and P5 (AT = 3, BT = 1) finished quickly with low waiting times (0 ms and 1 ms respectively) because their short remaining burst times let them jump ahead of longer jobs as soon as they arrived. P4 (AT = 0, BT = 6), the longest process, was repeatedly preempted — it ran first from t = 0–2 since it was the only process in the system, but was then pushed back every time a shorter process became ready, and only resumed and finished once P1, P2, P3, and P5 had all completed (t = 10–14). This gave P4 the highest waiting time (8 ms) in the set, illustrating that preemptive SJF can starve longer processes when shorter ones keep arriving.

Compared to non-preemptive SJF, this preemptive version reacts faster to newly arrived short processes (e.g., P5 preempted the queue almost immediately after arriving at t = 3), which generally lowers average waiting time further for short jobs at the expense of longer ones. Using a quantum of 2 ms instead of re-evaluating every 1 ms slightly delays preemption (a process can run up to 2 ms before the next check), which is why P4's first burst ran the full 2 ms before P2 and later arrivals were considered.

9. Conclusion

The preemptive SJF (SRTF) algorithm was successfully implemented and tested, giving an average TAT of 6.00 ms and an average WT of 3.20 ms for the given process set. The results confirm that preemptive SJF favors short processes and can significantly delay longer ones, trading fairness for lower average waiting time. This makes it well suited for workloads dominated by short, latency-sensitive jobs, but less appropriate where long processes must also receive timely CPU access.

10. Appendix: Python Source Code

The complete Python implementation used for this lab is listed below:

```
def sjf_preemptive(processes, quantum):
    n = len(processes)
    remaining_bt = [p[2] for p in processes]
    completed = 0
    current_time = 0
    is_done = [False] * n
    completion_time = [0] * n

    while completed != n:
        idx = -1
        min_rt = float('inf')

        for i in range(n):
            pid, at, bt = processes[i]
            if at <= current_time and not is_done[i] and remaining_bt[i] > 0:
                if remaining_bt[i] < min_rt:
                    min_rt= remaining_bt[i]
                    idx = i

        if idx == -1:
            current_time += 1
            continue

        run_time = min(quantum, remaining_bt[idx])
        remaining_bt[idx] -= run_time
        current_time += run_time

        if remaining_bt[idx] == 0:
            is_done[idx] = True
            completed += 1
            completion_time[idx] = current_time

    result = []
    for i in range(n):
        pid,at,bt=processes[i]
        ct=completion_time[i]
        tat=ct-at
        wt=            tat-bt
        result.append([pid,at,bt,ct,tat,wt])

    return result


processes = [
    ["P1", 4, 2],
    ["P2", 2, 2],
    ["P3", 1, 3],
    ["P4", 0, 6],
    ["P5", 3, 1]
]

quantum = 2
result = sjf_preemptive(processes, quantum)

result_sorted = sorted(result, key=lambda x: x[0])

print(f"{'Process':<8}{'AT':<5}{'BT':<5}{'CT':<5}{'TAT':<5}{'WT':<5}")
for row in result_sorted:
    pid, at, bt, ct, tat, wt = row
    print(f"{pid:<8}{at:<5}{bt:<5}{ct:<5}{tat:<5}{wt:<5}")

avg_tat = sum(r[4] for r in result) / len(result)
avg_wt = sum(r[5] for r in result) / len(result)
print(f"\nAverage TAT = {avg_tat:.2f}")
print(f"Average WT = {avg_wt:.2f}")
```

Githublink:https://github.com/Moshiur1143/Operating-System-Repository-23101143-/tree/main/Lab-05-Sjf-Preemptive